

Détection automatique du port des EPI sur chantier, SH17

Bloc 2, Projet 1 : rapport technique

Gautier

Juillet 2026

Introduction

Cadre et contexte

Les accidents du travail sur les chantiers de construction restent un enjeu majeur de sécurité. Selon l'INRS, les accidents graves dans le BTP impliquent souvent une absence ou un mauvais usage des Équipements de Protection Individuelle (EPI) : casques, gilets réfléchissants, lunettes, gants. Leur port est obligatoire (Code du travail), mais le contrôle reste aujourd'hui manuel, ponctuel et inefficace à l'échelle d'un chantier de plusieurs dizaines de travailleurs.

Objectifs

Concevoir un système automatisé capable de : détecter en temps réel le port des EPI à partir d'images ou de flux vidéo, déclencher des alertes pour les travailleurs non conformes, et s'intégrer dans les processus métiers existants (supervision, sécurité, gestion des chantiers).

Méthodologie générale

Le projet s'appuie sur le dataset **SH17** (8 099 images, 75 994 instances, 17 classes) et sur la famille de modèles **YOLO11** (détection en un seul passage), comparée à un transformeur de détection (**RT-DETR**). L'ensemble du pipeline (nettoyage, exploration, entraînement, évaluation, stratégie d'intégration) est documenté et reproductible dans `Applications/pipeline_SH17.ipynb`, dont ce rapport reprend et structure les résultats selon le plan attendu. Une contrainte a pesé sur la fin du projet : la perte d'accès à un serveur GPU dédié, qui a nécessité de poursuivre l'entraînement du modèle final en local (poste personnel) et de documenter honnêtement les conséquences sur la reproductibilité de certains résultats (détaillé section Évaluation comparative).

Préparation des données

Source

Dataset SH17 (Safe Human 17), récupéré automatiquement via `kagglehub` (`mugheesahmad/sh17-dataset-for-ppe-detection`), aucun téléchargement manuel requis. 17 classes d'EPI et de parties du corps (voir `CLAUDE.md` pour l'ordre exact des identifiants).

Split train / val / test

SH17 fournit un split train/val mais pas de split test. Un test set est créé en prélevant 15 % du train (`TEST_RATIO = 0.15`), graine fixée (`SEED = 42`) pour la reproductibilité du split et de l'entraînement.

Nettoyage

- **Images floues** : détection par variance du Laplacien (seuil recalibré sur la distribution réelle des scores de netteté du dataset), filtrage appliqué uniquement sur le train : le test et la validation restent représentatifs des conditions réelles rencontrées en production.
- **Annotations invalides** : boîtes englobantes hors image, dimensions nulles, doublons (détectés par IoU) supprimés ou corrigés.
- Chaque suppression est documentée et justifiée dans le notebook (section 1.3), avec le nombre d'images concernées à chaque étape.

Transformation

- **Redimensionnement / normalisation** : gérées nativement par Ultralytics (letterbox à 768 px, normalisation des pixels dans $[0,1]$), non recodées manuellement pour éviter une double normalisation. La résolution de 768 px (plutôt que la valeur par défaut 640) est motivée par l'exploration : environ 53 % des boîtes englobantes couvrent moins de 1 % de la surface de l'image, et les classes EPI critiques (`helmet` , `safety-vest`) sont à la fois rares et souvent petites.
- **Débruitage et recadrage manuel** : volontairement écartés (justification détaillée section 1.4 du notebook), un pré-traitement non justifié par l'exploration étant jugé plus risqué que bénéfique.
- **Augmentation** : mosaic natif YOLO (expose le modèle à plusieurs échelles) et transformations aléatoires ciblées (section 1.5 du notebook).

Analyse exploratoire

Répartition des classes et biais

Le ratio classe majoritaire / minoritaire dépasse 100:1 (`hands` contre `face-guard`). Les classes utiles à la conformité EPI sont précisément parmi les plus rares : `helmet` (927 instances au train), `safety-vest` (530), alors que `head` est très fréquente (11 985), cette asymétrie (beaucoup de têtes, peu de casques annotés) fragilise structurellement la détection des casques et motive l'axe d'analyse dédié (Axe B, voir plus bas).

Qualité des annotations et tailles de boîtes

Les statistiques de taille de boîtes (section 1.2.3 du notebook) montrent une forte proportion de petits objets, ce qui a directement motivé le choix de résolution d'entrée (768 px). L'inspection visuelle des exemples annotés (section 1.2.4) ne révèle pas d'artefacts systématiques justifiant un débruitage.

Insights métier

- Les zones du chantier les plus à risque (au sens du modèle) sont celles où les classes EPI critiques sont sous-représentées à l'entraînement : la performance du modèle sur `safety-vest` est la plus fragile de toutes les classes de conformité, ce qui doit être communiqué comme une limite connue aux équipes sécurité plutôt que masqué.
- Recommandation actionnable : la formation ciblée des équipes doit continuer de porter sur le port du gilet même si l'alerte automatique ne le couvre pas encore fermement (le système ne remplace pas la vigilance humaine sur ce point, cf. Stratégie d'intégration).

Modélisation Machine Learning

Choix des algorithmes

Quatre architectures comparées (au-delà du minimum de 3 demandé) : **yolo11s**, **yolo11m**, **yolo11l** (famille un seul passage, rapide) et **rtdetr-l** (transformeur de détection, plus lourd). Justification : YOLO11 offre le meilleur compromis vitesse / précision pour une détection en temps réel sur flux vidéo de chantier, avec un écosystème mature (format d'annotation, tracking natif). RT-DETR sert de point de comparaison pour vérifier qu'une architecture plus récente et plus lourde n'apporte pas un gain suffisant pour justifier son coût.

Entraînement

Modèle retenu : **yolo11l**, fine-tuné à résolution 768 px, hyperparamètres : `epochs=100`, `batch=4`, `optimizer=AdamW`, `lr0=8e-05`, `cos_lr=True`, `patience=20`, `seed=42`. Le taux d'apprentissage (`lr0=8e-05`) a été identifié par recherche d'hyperparamètres (`model.tune()`) sur un serveur GPU dédié (H100), puis réappliqué pour l'entraînement complet.

Continuité malgré la perte du serveur dédié. Le serveur H100 utilisé initialement n'étant plus disponible, l'entraînement complet du modèle final (100 époques) a été repris et mené à bien en local (poste personnel), avec un batch réduit à 4 pour tenir dans la mémoire GPU disponible (8 Go). Le modèle retenu, désigné `yolo11l_full-12` dans le dépôt, est le résultat de cette reprise en local. Ses performances (mAP@50-95 global 0.441) sont même légèrement supérieures à celles obtenues sur le serveur H100 avec les mêmes hyperparamètres (0.434-0.439), ce qui valide la démarche malgré la contrainte matérielle.

Gestion du sur-apprentissage

`patience=20` (arrêt anticipé), augmentation de données (mosaic + albumentations), et transfer learning depuis les poids pré-entraînés COCO plutôt qu'un entraînement from scratch.

Évaluation comparative des modèles

Comparaison des architectures (Axe A)

Modèle	mAP@50	mAP@50-95	GPU (ms/img)	CPU (ms/img)	Params (M)	Poids (Mo)
yolo11s	0.626	0.393	1.50	131.87	9.4	19.2
yolo11m	0.650	0.418	2.75	325.79	20.0	40.5
yolo11l	0.672	0.439	3.48	421.33	25.3	51.2
rtdetr-l	0.666	0.413	4.22	485.51	32.0	66.3
yolo11l-tuné (H100)	0.660	0.434	3.49	421.74	25.3	51.2

yolo11l domine RT-DETR sur le mAP tout en étant plus léger (25,3 M contre 32,0 M de paramètres) et plus rapide : RT-DETR est Pareto-dominé, il ne gagne sur aucun critère. Hypothèse : RT-DETR demande davantage de données et d'époques pour exprimer son potentiel. **yolo11m** constitue un compromis intermédiaire pertinent si le débit prime sur la précision (déploiement edge).

Note de méthode : ce tableau reprend des métriques obtenues lors d'un entraînement complet sur un serveur GPU aujourd'hui indisponible ; elles sont conservées telles quelles (les poids correspondants n'existent plus localement pour être recalculés) plutôt que recalculées avec des poids d'entraînement partiel non comparables.

Empreinte énergétique et robustesse (extension Axe A)

Contrairement au tableau comparatif principal (figé, serveur H100 disparu), cette extension a été **exécutée réellement en local** sur la carte disponible (RTX 4060 Laptop, 8 Go), y compris pour le modèle retenu.

Empreinte énergétique (proxy `nvidia-smi`, mesure de puissance instantanée du GPU pendant l'inférence, moyennée sur 30 images à 768 px, méthode approximative documentée comme telle dans le notebook) :

Modèle	GFLOPs	Puissance moy. (W)	Énergie (J/img)	gCO2/img (estimation)
yolo11s	21.3	14.7	5.579	0.07748
yolo11m	67.7	17.3	4.539	0.06305
yolo11l	86.6	16.8	4.530	0.06291
rtdetr-l	103.5	21.0	5.985	0.08312
yolo11l_full-12 (retenu)	86.6	17.8	4.633	0.06435

Les 5 architectures ont un profil de consommation proche (14,7 à 21,0 W en moyenne) : le choix de `yolo11l` ne se justifie pas par une empreinte énergétique meilleure (elle est comparable aux autres YOLO11), mais par le compromis mAP / rappel sur les classes EPI rares établi dans la comparaison principale. `rtdetr-l` est la configuration la plus gourmande, cohérent avec son nombre de paramètres plus élevé.

Robustesse (sous-ensemble de 200 images de test dégradées : faible luminosité + occlusion synthétique, modèle retenu uniquement : les architectures “smoke” sous-entraînées ne permettent pas une comparaison de delta significative) :

	mAP@50	mAP@50-95
Baseline (test propre)	0.667	0.441
Dégradé (faible luminosité + occlusion)	0.495	0.308
Delta	-0.172	-0.133

Le delta est net (mAP@50-95 chute d'environ 30 % en relatif). C'est une **limite réelle** à documenter : l'augmentation appliquée à l'entraînement (mosaic + transformations aléatoires) ne couvre pas suffisamment ce niveau de dégradation. Ce résultat renforce la recommandation de la feuille de route de déploiement (Stratégie d'intégration) : une phase de test terrain de 4 à 6

semaines couvrant des conditions météo/luminosité variées est nécessaire avant toute généralisation, et un renforcement de l'augmentation (luminosité, occlusion) est une piste d'amélioration à prioriser avant un déploiement en extérieur non contrôlé.

Focus 3 classes de conformité (Axe B)

Hypothèse : en retirant les 14 classes annexes, le modèle ne dilue plus sa capacité entre `person` / `hands` et les 3 classes de conformité ; attente réaliste (posée avant expérience) : `head` a déjà peu de marge (0.730 mAP@50-95), `helmet` distinctif devrait progresser modérément, `safety-vest` reste le plus dur car son déséquilibre interne (head 8211 contre vest 372, environ 22:1) persiste même après remapping.

Protocole : `yolo11l` ré-entraîné sur les 3 classes remappées (`helmet` / `head` / `safety-vest` , ré-indexées 0/1/2), mêmes hyperparamètres que le modèle 17-classes retenu à l'exception du nombre d'époques (40 au lieu de 100, faute de serveur GPU dédié). Le remapping écarte les images ne contenant aucune des 3 classes cibles, soit environ 19 % du train (4445 images contre 5508). Comparaison **indicative** face au modèle 17-classes final (entraîné 100 époques) : une partie de l'écart peut venir du nombre d'époques, pas seulement du nombre de classes.

Classe	mAP@50-95 (17-cls)	mAP@50-95 (3-cls)	Δ
helmet	0.499	0.412	-0.087
head	0.730	0.696	-0.034
safety-vest	0.307	0.317	+0.010
Moyenne (3 classes)	0.512	0.475	-0.037

Résultat contre-intuitif, mais pas uniforme. Restreindre l'entraînement aux 3 classes dégrade en moyenne leurs performances (-0.037 de mAP@50-95), avec un recul net sur `helmet` (-17 % en relatif) et `head` (-5 %). `safety-vest` est le seul cas où la restriction aide légèrement (+0.010 sur le mAP@50-95, +0.062 sur le mAP@50) : la précision progresse (0.587 → 0.617) mais le rappel recule (0.606 → 0.541), soit moins de détections mais plus fiables. Trois mécanismes expliquent ce bilan globalement négatif :

1. **Perte du partage de features inter-classes** (mécanisme principal, explique le recul de `helmet` et `head`) : YOLO apprend un backbone commun, où voir des milliers de `person`, `head`, `hands`, `face` apprend au réseau à localiser un humain et ses parties, et un casque se détecte relativement à une tête. Supprimer les 14 classes ampute ce contexte.
2. **Moins de données d'entraînement** (19 % d'images écartées par le remapping).
3. **Le déséquilibre interne persiste** (head/safety-vest $\approx$ 22:1) : l'Axe B retire la dilution externe mais pas ce déséquilibre. Le léger mieux sur `safety-vest` (précision en hausse) suggère que retirer le bruit des 14 classes annexes aide un peu le modèle à être plus sélectif sur cette classe difficile, sans compenser la perte de rappel.

Recommandation actionable : pour un détecteur d'EPI fondé sur YOLO, conserver les classes anatomiques (`person` , `head` , `face`) comme contexte est préférable à un entraînement restreint : le gain marginal et incertain sur `safety-vest` ne compense pas la perte sur `helmet` / `head` , les deux classes qui fondent l'alerte ferme de l'application.

Modèle final retenu

`yolo11l_full-12` , évalué sur le jeu de test (971 images) :

Classe	n_train	Précision	Rappel	F1	mAP@50	mAP@50-95
person	9509	0.908	0.917	0.913	0.948	0.784
head	8211	0.926	0.926	0.926	0.940	0.730
face	6068	0.951	0.917	0.934	0.947	0.709
hands	10823	0.902	0.868	0.885	0.908	0.642
ear	5224	0.901	0.846	0.873	0.858	0.551
helmet	652	0.858	0.620	0.720	0.733	0.499
gloves	1938	0.799	0.687	0.739	0.745	0.487
shoes	3064	0.790	0.692	0.738	0.731	0.442
face-mask	428	0.758	0.517	0.615	0.557	0.383
glasses	1344	0.749	0.676	0.711	0.684	0.382
medical-suit	94	0.578	0.600	0.589	0.590	0.377
ear-mufs	223	0.810	0.500	0.618	0.616	0.342
safety-vest	372	0.587	0.606	0.597	0.481	0.307
tool	3121	0.603	0.458	0.521	0.466	0.261
face-guard	89	0.824	0.448	0.581	0.498	0.251
foot	514	0.540	0.342	0.419	0.362	0.183
safety-suit	164	0.302	0.432	0.355	0.280	0.176

mAP@50 global : 0.667, mAP@50-95 global : 0.441.

La performance par classe est fortement corrélée à la fréquence d'entraînement, avec deux exceptions notables : `helmet` (652 instances) surperforme sa fréquence grâce à sa distinctivité visuelle (couleur vive, forme constante), tandis que `tool` (3121 instances, pourtant fréquent) reste bas (0.261) car visuellement hétérogène. Le modèle est globalement conservateur (précision > rappel), sauf sur `safety-vest` , `medical-suit` et `safety-suit` où le rappel dépasse légèrement la précision. En contexte sécurité, cette asymétrie précision/rappel justifie d'abaisser le seuil de confiance côté casque (privilégier le rappel) plutôt que de le laisser au seuil qui maximiserait le F1.

Limites documentées

- `safety-vest` (mAP@50-95 0.307) : rappel insuffisant pour fonder une alerte ferme, reste un indice indicatif dans l'application.
- Pas de tracking multi-frame : chaque alerte est évaluée frame par frame, sans persistance par individu (piste d'amélioration identifiée en phase 3 de la feuille de route de déploiement).
- Le dataset SH17 n'est pas exclusivement issu du BTP, ce qui peut créer un écart entre performance mesurée et performance en conditions réelles de chantier.

Validation

Modèle validé en interne sur la base des métriques ci-dessus (rappel prioritaire sur `helmet / head`, seul couple fondant l'alerte ferme). La validation par les parties prenantes métier (équipe sécurité, direction) est prévue en phase 1 de la feuille de route de déploiement (voir Stratégie d'intégration).

Stratégie d'intégration

Voir [STRATEGIE_INTEGRATION.md](#) pour le détail complet (cas d'usage prioritaires, impact métier, analyse SWOT, feuille de route de déploiement en 3 phases). Synthèse :

- **Priorisation** : casque (`helmet / head`) en premier, seul EPI fondant une alerte ferme, cohérent avec le risque le plus grave (traumatisme crânien) et avec la fiabilité relative du modèle sur ce couple. Gilet en indice non bloquant. Gants/lunettes hors périmètre de l'alerte automatique pour l'instant.
- **Déploiement** : chantier pilote unique, zone d'entrée/sortie fixe, 4 à 6 semaines de test avant généralisation.
- **Freins identifiés** : acceptabilité du contrôle vidéo par les travailleurs, fiabilité perçue liée au taux de faux positifs, contraintes d'infrastructure chantier (connectivité, alimentation, poussière/éclairage), responsabilité juridique (le système est une aide à la décision, pas un substitut au contrôle humain).

Conclusion

Bilan

Le système développé détecte les 17 classes du dataset SH17 et fonde une alerte de non-conformité sur le couple casque/tête, la classe la plus fiable du modèle. Le modèle retenu (`yolo11l_full-12`) a été mené à son terme malgré la perte d'un serveur GPU dédié en cours de

projet, avec des performances finales égales ou légèrement supérieures à celles obtenues initialement sur serveur. L'application Streamlit fournit une démonstration fonctionnelle (image, vidéo, tableau de bord) avec une logique de conformité synchronisée avec le notebook.

Limites

Rappel insuffisant sur `safety-vest` pour une alerte ferme ; pas de validation de robustesse sur chantier réel (conditions dégradées) au-delà du proxy mesuré en local ; pas de tracking multi-frame ; comparaison Axe B menée avec un protocole d'entraînement réduit (40 époques au lieu de 100) faute de ressources GPU dédiées, donc indicative plutôt que strictement rigoureuse.

Perspectives

Ré-entraînement avec des données terrain réelles (phase 3 de la feuille de route), étude du tracking multi-frame pour réduire le bruit des alertes, étude d'un modèle plus léger pour un déploiement edge si la contrainte serveur central se confirme durablement.